

AUTH SYSTEM REACT

React • Node.js • MongoDB • JWT

Technical Documentation — v1.0.0 — May 2026

Version	1.0.0
Auth Strategy	JWT Access Token (15 min) + HttpOnly Refresh Cookie (7 days)
Backend	Node.js 18 + Express 4 + Mongoose 8
Frontend	React 18 + TypeScript + Vite + Tailwind CSS
Database	MongoDB (embedded refresh token sub-documents)

1. Project Overview

Auth System React is a full-stack authentication application. The backend is an Express REST API managing JWT tokens and MongoDB user records. The frontend is a React 18 TypeScript SPA that manages auth state via React Context and communicates with the API through a fetch wrapper that handles silent token refresh automatically.

The system uses a dual-token pattern: a short-lived access token returned in the JSON body, and a long-lived refresh token stored as an HttpOnly cookie that JavaScript cannot read, protecting against XSS-based session hijacking.

1.1 Key Features

- JWT dual-token auth: 15-minute access token + 7-day HttpOnly refresh cookie
- Automatic silent token refresh — apiClient retries failed requests transparently
- Refresh token rotation — old token invalidated on every refresh call
- Token reuse detection — all sessions revoked when a consumed token is replayed
- Brute-force protection — account locked 30 min after 5 failed login attempts
- Rate limiting — 10 req/15 min on login & register; 30 req/15 min on refresh
- Multi-device session management with IP and User-Agent metadata
- Role-based access control — user | admin roles via restrictTo() middleware
- Server-side validation via express-validator with per-field error responses
- Client-side password strength indicator on the registration form

1.2 Technology Stack

Layer	Technology	Purpose
Frontend	React 18 + TypeScript + Vite	SPA, auth state, page routing
Styling	Tailwind CSS 3	Utility-first CSS framework
Backend	Node.js + Express 4	REST API server
Database	MongoDB + Mongoose 8	User & session persistence
Auth	jsonwebtoken + bcryptjs	Token signing & password hashing
Security	helmet + cors + express-rate-limit	HTTP hardening & DDoS protection
Validation	express-validator	Server-side input sanitisation
Dev Tools	nodemon + morgan	Hot reload & HTTP request logging

2. Architecture

2.1 Directory Structure

```
auth-system-react/
├── backend/
│   ├── config/
│   │   ├── db.js           # Mongoose connection with reconnect events
│   │   └── jwt.js          # generateTokenPair, verifyAccessToken,
│   │                       verifyRefreshToken
│   ├── controllers/
│   │   └── authController.js # register, login, refresh, logout, logoutAll, getMe,
│   │                       getSessions
│   ├── middleware/
│   │   ├── authMiddleware.js # protect (Bearer JWT) + restrictTo (RBAC)
│   │   └── validateMiddleware.js # validateRegister, validateLogin chains
│   ├── models/
│   │   └── User.js          # Mongoose schema + bcrypt + refresh token methods
│   ├── routes/
│   │   └── authRoutes.js    # Route definitions + rate limiters
│   └── server.js            # Express bootstrap, middleware chain, static serve
└── frontend/src/
    ├── context/AuthContext.tsx # Global auth state + login/register/logout
    ├── pages/LoginPage.tsx
    ├── pages/RegisterPage.tsx
    ├── pages/DashboardPage.tsx
    ├── components/PasswordInput.tsx
    ├── utils/apiClient.ts      # Fetch wrapper with auto-refresh
    ├── utils/tokenManager.ts   # localStorage access token helpers
    └── App.tsx                 # Root component + in-state page router
```

2.2 Authentication Flow

1. User submits credentials via LoginPage form
2. AuthContext.login() calls apiClient.post('/auth/login', credentials)
3. Server validates, bcrypt-compares password, checks lock status
4. On success: generates access token (JWT, 15 min) + refresh token (JWT, 7 days)
5. Access token returned in JSON body; refresh token set as HttpOnly SameSite=Strict cookie
6. Frontend stores access token in localStorage via TokenManager
7. All API calls include Authorization: Bearer <token> header
8. On 401 + code: TOKEN_EXPIRED, apiClient calls POST /api/auth/refresh
9. Server rotates refresh token (old revoked, new issued) and returns new access token
10. TokenManager updated; original request retried — user sees no interruption

SECURITY	Token reuse detection: presenting an already-consumed refresh token causes the server to wipe ALL sessions for the user, forcing re-login on all devices.
-----------------	---

3. Backend Documentation

3.1 Environment Variables

Variable	Default	Description
PORT	5000	HTTP server port
NODE_ENV	development	Controls CSP, morgan, stack traces in errors
MONGODB_URI	(required)	Full MongoDB connection string
JWT_ACCESS_SECRET	(required)	Secret for signing 15-min access tokens
JWT_REFRESH_SECRET	(required)	Secret for signing 7-day refresh tokens
JWT_ACCESS_EXPIRY	15m	Access token lifetime (e.g. 15m, 1h)
JWT_REFRESH_EXPIRY	7d	Refresh token lifetime (e.g. 7d, 30d)
CLIENT_URL	http://localhost:3000	Allowed CORS origin

TIP

Generate strong 64-byte secrets: `node -e "console.log(require('crypto').randomBytes(64).toString('hex'))"`

3.2 User Model (MongoDB)

File: backend/models/User.js — Mongoose schema with embedded refresh token sub-documents and instance methods for password comparison, brute-force tracking, token cleanup, and safe serialisation.

Field	Type	Required	Notes
name	String	Yes	2-50 chars, trimmed
email	String	Yes	Unique, lowercase, regex-validated
password	String	Yes	Min 8 chars, bcrypt(12) hashed, select:false
role	String	No	Enum: user admin. Default: user
isActive	Boolean	No	Default: true. Inactive users cannot login
refreshTokens	Array	No	Sub-docs: token, expiresAt, userAgent, ip
lastLogin	Date	No	Updated on every successful login
loginAttempts	Number	No	Incremented on each failed login
lockUntil	Date	No	Set to now+30 min after 5 failures

Instance Methods

- `comparePassword(candidate)` — `bcrypt.compare`, returns boolean
- `incLoginAttempts()` — increments counter; sets `lockUntil` after 5 failures

- `cleanExpiredTokens()` — filters sub-docs where `expiresAt < now`
- `toPublicJSON()` — safe serialisation omitting password and refreshTokens
- `isLocked (virtual)` — true when `lockUntil` is in the future

3.3 API Routes

Method	Path	Guard	Description
POST	/api/auth/register	Public	Create account. 10 req/15 min rate limit.
POST	/api/auth/login	Public	Login. Returns access token + sets cookie.
POST	/api/auth/refresh	Cookie	Rotate refresh token. 30 req/15 min.
POST	/api/auth/logout	Bearer	Invalidate current session's refresh token.
POST	/api/auth/logout-all	Bearer	Wipe all sessions across all devices.
GET	/api/auth/me	Bearer	Return authenticated user profile.
GET	/api/auth/sessions	Bearer	List active sessions with metadata.
GET	/api/health	Public	Health check: returns status + timestamp.

3.4 Middleware

protect (authMiddleware.js)

Reads the Bearer token from the Authorization header (or accessToken cookie as fallback). Verifies JWT signature and claims (issuer: auth-system, audience: auth-system-client). Loads the user from MongoDB, attaches to `req.user`. Returns 401 + code: `TOKEN_EXPIRED` on expiry so the frontend can trigger a silent refresh.

restrictTo(...roles)

Factory middleware comparing `req.user.role` against the allowed roles array. Returns 403 if unauthorized. Example: `router.get('/admin', protect, restrictTo('admin'), handler)`.

validateRegister / validateLogin

express-validator chains: name (2-50 chars), email (valid format, normalised), password (min 8 chars, must contain uppercase + lowercase + digit). Returns 422 with errors: `[{ field, message }]` on failure.

3.5 Security Summary

Measure	Implementation
Helmet	Sets 14 security HTTP headers (X-Frame-Options, HSTS, CSP, etc.)

CORS	Origin restricted to CLIENT_URL, credentials: true, SameSite: strict
Rate Limiting	10 req/15 min on auth; 30 req/15 min on refresh (express-rate-limit)
Brute Force	5 failures => 30-min lock. Counter reset on successful login.
Password Hashing	bcrypt with salt factor 12 (prevents rainbow table attacks)
Token Rotation	Refresh token consumed and re-issued on every /refresh call
Reuse Detection	All sessions wiped when a spent token is replayed
HttpOnly Cookie	Refresh token unreachable by JavaScript (XSS-resistant)
Payload Limit	express.json({ limit: '10kb' }) prevents large-payload DoS
Session Cap	refreshTokens capped at 5; oldest evicted on overflow

4. Frontend Documentation

4.1 State Management (AuthContext)

File: `src/context/AuthContext.tsx`. Central source of truth for authentication state and actions.

Export	Type	Description
<code>user</code>	<code>User null</code>	Authenticated user: id, name, email, role, lastLogin, createdAt
<code>accessToken</code>	<code>string null</code>	Current JWT held in React state
<code>page</code>	<code>login register dashboard</code>	Active page; controls which component App renders
<code>loading</code>	<code>boolean</code>	True while initial session validation is in progress
<code>login(email, pw)</code>	<code>() => Promise</code>	POST <code>/auth/login</code> , stores token, navigates to dashboard
<code>register(name,email,pw)</code>	<code>() => Promise</code>	POST <code>/auth/register</code> , auto-logs in on success
<code>logout()</code>	<code>() => Promise</code>	POST <code>/auth/logout</code> , clears state + localStorage
<code>logoutAll()</code>	<code>() => Promise</code>	POST <code>/auth/logout-all</code> , wipes all sessions

On mount, `AuthContext` reads `localStorage` for a stored access token. If present, calls `GET /api/auth/me` to validate it. Valid response navigates to dashboard; any failure clears storage and shows login.

4.2 API Client (`apiClient.ts`)

- Attaches Authorization: Bearer <token> header automatically on every request
- Sets credentials: 'include' so the HttpOnly refresh cookie is always sent
- On 401 + code === `TOKEN_EXPIRED`: calls `/api/auth/refresh`, updates token in `localStorage`, retries original request (`_retry` flag prevents infinite loops)
- If refresh fails: clears `localStorage` and reloads the page
- Convenience methods: `apiClient.get(url)`, `.post(url, body)`, `.delete(url)`

4.3 Token Manager (`tokenManager.ts`)

Single module owning all `localStorage` access for auth data:

- `setAccessToken` / `getAccessToken` / `clearAccessToken`
- `setUser` / `getUser` (JSON-serialised) / `clearUser`
- `clear()` — removes both keys atomically

4.4 Pages

LoginPage.tsx

Email + password form with error banner and loading spinner. Delegates to AuthContext.login(). On failure, displays the server error message. Link to RegisterPage via setPage('register').

RegisterPage.tsx

Name + email + password form. Real-time password strength meter scores across 4 dimensions (length ≥ 8 , uppercase, digit, special char). Per-field API validation errors mapped below each input. On success, auto-logs the user in.

DashboardPage.tsx

Protected area with four panels:

- User Profile — avatar initial, name, email, role badge, user ID, join date
- Active Sessions — fetched from GET /api/auth/sessions; shows User-Agent, IP, expiry per device
- JWT Access Token — full token displayed with copy-to-clipboard; live expiry decoded from payload; token lifetime breakdown
- Security Features — visual summary of Token Rotation, Brute Force, Rate Limiting, Reuse Detection

4.5 Frontend Dependencies

Package	Version	Purpose
react	^18.3.1	Core UI library
react-dom	^18.3.1	DOM renderer
typescript	^5.7.2	Static type checking
vite	^6.0.5	Build tool & dev server with HMR
tailwindcss	^3.4.17	Utility CSS framework
@vitejs/plugin-react	^4.3.4	React fast-refresh for Vite

5. Setup & Running Locally

5.1 Prerequisites

- Node.js >= 18.x and npm >= 9.x
- MongoDB (local instance on port 27017 or a MongoDB Atlas URI)

5.2 Backend

```
cd backend
npm install
cp .env.example .env
# Fill in MONGODB_URI, JWT_ACCESS_SECRET, JWT_REFRESH_SECRET
npm run dev          # nodemon; restarts on file changes
```

5.3 Frontend

```
cd frontend
npm install
npm run dev          # Vite dev server on http://localhost:3000
```

Vite proxies `/api/*` to the Express server in dev mode. Configure in `vite.config.ts` if the backend runs on a different port:

```
server: { proxy: { '/api': 'http://localhost:5000' } }
```

5.4 Production Build

```
cd frontend && npm run build  # outputs to frontend/dist/
# Set NODE_ENV=production in backend .env
cd backend && npm start        # serves API + static frontend
```

Express serves `frontend/dist/` as static files and provides the SPA fallback (`app.get('*', ...sendFile)`) so browser refreshes return `index.html` correctly.

6. API Reference

POST /api/auth/register

Request Body

```
{ "name": "Jane Doe", "email": "jane@example.com", "password": "Secret123" }
```

201 Success

```
{ "success": true, "message": "Registration successful!",  
  "data": { "user": { ...publicFields }, "accessToken": "eyJ...", "expiresIn": "15m" } }
```

Errors

- 409 — Email already registered
- 422 — Validation failed: { errors: [{ field, message }] }
- 429 — Rate limit exceeded

POST /api/auth/login

Request Body

```
{ "email": "jane@example.com", "password": "Secret123" }
```

200 Success

```
{ "success": true, "data": { "user": { ...publicFields }, "accessToken": "eyJ..." } }
```

Sets cookie: refreshToken=...; HttpOnly; SameSite=Strict; Max-Age=604800

Errors

- 401 — Invalid credentials (remaining attempts in message)
- 423 — Account locked (minutes remaining in message)
- 403 — Account deactivated

POST /api/auth/refresh

Reads refreshToken from HttpOnly cookie automatically.

200 Success

```
{ "success": true, "data": { "accessToken": "eyJ...", "expiresIn": "15m" } }
```

Errors

- 401 — Token missing, invalid, or expired
- 401 — Token reuse detected: all sessions revoked, please re-login

GET /api/auth/me

Requires: Authorization: Bearer <accessToken>

200 Success

```
{ "success": true, "data": { "user": { "id": "...", "name": "...", "email": "...",  
  "role": "user", "isActive": true, "lastLogin": "...", "createdAt": "..." } } }
```

GET /api/auth/sessions

Requires: Authorization: Bearer <accessToken>

200 Success

```
{ "success": true, "data": { "sessions": [{ "createdAt": "...", "expiresAt": "...",  
  "userAgent": "Mozilla/5.0...", "ip": "127.0.0.1" }], "count": 1 } }
```

POST /api/auth/logout

Requires: Authorization: Bearer <accessToken>. Removes the current session's refresh token. Other sessions remain active. Clears the cookie on the current device.

POST /api/auth/logout-all

Requires: Authorization: Bearer <accessToken>. Sets user.refreshTokens = [] in MongoDB, invalidating every active session. Clears the cookie on the current device.

7. Security Considerations

7.1 Token Storage

WARNING The access token is stored in `localStorage`, which is accessible to JavaScript and therefore vulnerable to XSS. Mitigate by enforcing a strict Content Security Policy and avoiding `innerHTML`-based rendering.

The refresh token is exclusively in an `HttpOnly SameSite=Strict` cookie, making it inaccessible to JavaScript and protected from both XSS and CSRF attacks.

7.2 Password Policy

Passwords must be at least 8 characters with at least one uppercase letter, one lowercase letter, and one digit. `bcrypt` at cost factor 12 makes offline brute-force attacks computationally expensive.

7.3 Production Hardening Checklist

- Set `NODE_ENV=production` — disables stack traces in error responses
 - Deploy over HTTPS — required for the Secure cookie attribute
 - Tighten the Content-Security-Policy header (Helmet sets permissive defaults)
 - Store JWT secrets in a secrets manager (AWS Secrets Manager, HashiCorp Vault)
 - Enable MongoDB authentication and restrict access with IP allowlists
 - Consider storing the access token in a React memory variable (`useState`) instead of `localStorage` for stricter XSS isolation
 - Implement proactive token refresh (`setTimeout` before expiry) to avoid mid-action session expiry
 - Add HTTPS-only redirect and HSTS preloading
-